arXiv:1611.01462v3 [cs.LG] 11 Mar 2017

TYING WORD VECTORS AND WORD CLASSIFIERS: A LOSS FRAMEWORK FOR LANGUAGE MODELING

Hakan Inan, Khashayar Khosravi
Stanford University
Stanford, CA, USA
{inanh,khosravi}@stanford.edu

Richard Socher
Salesforce Research
Palo Alto, CA, USA
rsocher@salesforce.com

ABSTRACT

Recurrent neural networks have been very successful at predicting sequences of words in tasks such as language modeling. However, all such models are based on the conventional classification framework, where the model is trained against one-hot targets, and each word is represented both as an input and as an output in isolation. This causes inefficiencies in learning both in terms of utilizing all of the information and in terms of the number of parameters needed to train. We introduce a novel theoretical framework that facilitates better learning in language modeling, and show that our framework leads to tying together the input embedding and the output projection matrices, greatly reducing the number of trainable variables. Our framework leads to state of the art performance on the Penn Treebank with a variety of network models.

1 INTRODUCTION

Neural network models have recently made tremendous progress in a variety of NLP applications such as speech recognition (Irie et al., 2016), sentiment analysis (Socher et al., 2013), text summarization (Rush et al., 2015; Nallapati et al., 2016), and machine translation (Firat et al., 2016).

Despite the overwhelming success achieved by recurrent neural networks in modeling long range dependencies between words, current recurrent neural network language models (RNNLM) are based on the conventional classification framework, which has two major drawbacks: First, there is no assumed metric on the output classes, whereas there is evidence suggesting that learning is improved when one can define a natural metric on the output space (Frogner et al., 2015). In language modeling, there is a well established metric space for the outputs (words in the language) based on word embeddings, with meaningful distances between words (Mikolov et al., 2013; Pennington et al., 2014). Second, in the classical framework, inputs and outputs are considered as isolated entities with no semantic link between them. This is clearly not the case for language modeling, where inputs and outputs in fact live in identical spaces. Therefore, even for models with moderately sized vocabularies, the classical framework could be a vast source of inefficiency in terms of the number of variables in the model, and in terms of utilizing the information gathered by different parts of the model (e.g. inputs and outputs).

In this work, we introduce a novel loss framework for language modeling to remedy the above two problems. Our framework is comprised of two closely linked improvements. First, we augment the classical cross-entropy loss with an additional term which minimizes the KL-divergence between the model's prediction and an estimated target distribution based on the word embeddings space. This estimated distribution uses knowledge of word vector similarity. We then theoretically analyze this loss, and this leads to a second and synergistic improvement: tying together two large matrices by reusing the input word embedding matrix as the output classification matrix. We empirically validate our theory in a practical setting, with much milder assumptions than those in theory. We also find empirically that for large networks, most of the improvement could be achieved by only reusing the word embeddings.

We test our framework by performing extensive experiments on the Penn Treebank corpus, a dataset widely used for benchmarking language models (Mikolov et al., 2010; Merity et al., 2016). We demonstrate that models trained using our proposed framework significantly outperform models

arXiv:1611.01462v3 [cs.LG] 11 Mar 2017

词向量与词分类器绑定框架：一种用于语言建模的轻量级框架

哈坎·伊南，卡沙亚尔·霍斯拉维，斯坦福大学，美国加利福尼亚州斯坦福市{inanh,khosravi}@stanford.edu

Richard Socher，Salesforce研究部门，美国加利福尼亚州帕洛阿尔托市，rsocher@salesforce.com

摘要

在语言建模等任务中，循环神经网络在预测单词序列方面取得了极为优异的成果。然而，所有这类模型都基于传统的分类框架，即模型需以独热编码目标作为训练依据，且每个单词既被单独视为输入，又被单独视为输出。这种设计不仅无法充分利用所有信息，还会增加训练所需的参数数量，从而导致学习效率低下。我们提出了一种全新的理论框架，有助于提升语言建模的学习效果，并证明了该框架能够将输入嵌入矩阵与输出投影矩阵相结合，从而大幅减少需要训练的变量数量。通过多种网络模型在宾州树库上的测试，我们的框架展现出了业界领先的性能。

1 引言

Neural network models have recently made tremendous progress in a variety of NLP applications such as speech recognition (Irie et al., 2016), sentiment analysis (Socher et al., 2013), text summarization (Rush et al., 2015; Nallapati et al., 2016), and machine translation (Firat et al., 2016).

Despite the overwhelming success achieved by recurrent neural networks in modeling long range dependencies between words, current recurrent neural network language models (RNNLM) are based on the conventional classification framework, which has two major drawbacks: First, there is no assumed metric on the output classes, whereas there is evidence suggesting that learning is improved when one can define a natural metric on the output space (Frogner et al., 2015). In language modeling, there is a well established metric space for the outputs (words in the language) based on word embeddings, with meaningful distances between words (Mikolov et al., 2013; Pennington et al., 2014). Second, in the classical framework, inputs and outputs are considered as isolated entities with no semantic link between them. This is clearly not the case for language modeling, where inputs and outputs in fact live in identical spaces. Therefore, even for models with moderately sized vocabularies, the classical framework could be a vast source of inefficiency in terms of the number of variables in the model, and in terms of utilizing the information gathered by different parts of the model (e.g. inputs and outputs).

在本研究中，我们为语言建模引入了一种全新的损失框架，旨在解决上述两个问题。该框架由两项紧密关联的改进措施构成。首先，我们在传统的交叉熵损失基础上增加了额外的项，该项用于最小化模型预测结果与基于词嵌入空间估算得到的目标分布之间的KL散度；这一估算目标分布是利用词向量相似性知识得出的。随后我们对这种损失机制进行了理论分析，进而提出了第二项具有协同效应的改进方案：通过复用输入端的词嵌入矩阵作为输出端的分类矩阵，将两个庞大的矩阵相互关联起来。我们在实际应用环境中对这一理论进行了验证，所采用的假设条件比理论分析中的要宽松得多。实验还表明，对于大型网络而言，仅通过复用词嵌入即可实现大部分性能提升。

我们通过在广泛用于语言模型基准测试的Penn Treebank语料库上开展大量实验来验证我们的框架（Mikolov等人，2010年；Merity等人，2016年）。实验结果表明，使用我们所提出的框架训练得到的模型性能显著优于传统模型

trained using the conventional framework. We also perform experiments on the newly introduced Wikitext-2 dataset (Merity et al., 2016), and verify that the empirical performance of our proposed framework is consistent across different datasets.

2 BACKGROUND: RECURRENT NEURAL NETWORK LANGUAGE MODEL

In any variant of recurrent neural network language model (RNNLM), the goal is to predict the next word indexed by t in a sequence of one-hot word tokens $(y_1^*, \dots y_N^*)$ as follows:

$$x_t = Ly_{t-1}^*, \tag{2.1}$$
$$h_t = f(x_t, h_{t-1}), \tag{2.2}$$
$$y_t = \text{softmax}\left(Wh_t + b\right). \tag{2.3}$$

The matrix $L \in \mathbb{R}^{d_x \times |V|}$ is the word embedding matrix, where d_x is the word embedding dimension and $|V|$ is the size of the vocabulary. The function $f(.,.)$ represents the recurrent neural network which takes in the current input and the previous hidden state and produces the next hidden state. $W \in \mathbb{R}^{|V| \times d_h}$ and $b \in \mathbb{R}^{|V|}$ are the the output projection matrix and the bias, respectively, and d_h is the size of the RNN hidden state. The $|V|$ dimensional y_t models the discrete probability distribution for the next word.

Note that the above formulation does not make any assumptions about the specifics of the recurrent neural units, and f could be replaced with a standard recurrent unit, a gated recurrent unit (GRU) (Cho et al., 2014), a long-short term memory (LSTM) unit (Hochreiter & Schmidhuber, 1997), etc. For our experiments, we use LSTM units with two layers.

Given y_t for the tth example, a loss is calculated for that example. The loss used in the RNNLMs is almost exclusively the cross-entropy between y_t and the observed one-hot word token, y_t^*:

$$J_t = \text{CE}(y_t^* \parallel y_t) = -\sum_{i \in |V|} y_{t,i}^* \log y_{t,i}. \tag{2.4}$$

We shall refer to y_t as the model prediction distribution for the tth example, and y_t^* as the empirical target distribution (both are in fact conditional distributions given the history). Since cross-entropy and *Kullback-Leibler* divergence are equivalent when the target distribution is one-hot, we can rewrite the loss for the tth example as

$$J_t = \text{D}_{KL}(y_t^* \parallel y_t). \tag{2.5}$$

Therefore, we can think of the optimization of the conventional loss in an RNNLM as trying to minimize the distance[1] between the model prediction distribution (y) and the empirical target distribution (y^*), which, with many training examples, will get close to minimizing distance to the actual target distribution. In the framework which we will introduce, we utilize *Kullback-Leibler* divergence as opposed to cross-entropy due to its intuitive interpretation as a distance between distributions, although the two are not equivalent in our framework.

3 AUGMENTING THE CROSS-ENTROPY LOSS

We propose to augment the conventional cross-entropy loss with an additional loss term as follows:

$$\hat{y}_t = \text{softmax}\left(Wh_t/\tau\right), \tag{3.1}$$
$$J_t^{aug} = \text{D}_{KL}(\tilde{y}_t \parallel \hat{y}_t), \tag{3.2}$$
$$J_t^{tot} = J_t + \alpha J_t^{aug}. \tag{3.3}$$

In above, α is a hyperparameter to be adjusted, and $\hat{y}_t$ is almost identical to the regular model prediction distribution y_t with the exception that the logits are divided by a temperature parameter τ. We define $\tilde{y}_t$ as some probability distribution that estimates the true data distribution (conditioned on the word history) which satisfies $\mathbb{E}\tilde{y}_t = \mathbb{E}y_t^*$. The goal of this framework is to minimize the

[1] We note, however, that *Kullback-Leibler* divergence is not a valid distance metric.

该框架是使用传统方法训练得到的。我们还在新引入的Wikitext-2数据集上进行了实验（Merity等人，2016年），并验证了所提出的框架在不同数据集上的实际性能具有稳定性。

2 背景：循环神经网络语言模型

在各类循环神经网络语言模型（RNNLM）中，其目标都是如下所示，预测由一热词标记序列（y_1^*, $\dots y_N^*$）中的 t 所索引的下一个词：

$$x_t = Ly_{t-1}^*, \tag{2.1}$$
$$h_t = f(x_t, h_{t-1}), \tag{2.2}$$
$$y_t = \text{softmax}\left(Wh_t + b\right). \tag{2.3}$$

矩阵 $L \in \mathbb{R}^{d_x \times |V|}$ 即为词嵌入矩阵，其中 d_x 代表词嵌入的维度，$|V|$ 则表示词汇表的大小。函数 $f(.,.)$代表RNN，它接收当前的输入与之前的隐藏状态，进而生成下一个隐藏状态。$W \in \mathbb{R}^{|V| \times d_h}$ 和 $b \in \mathbb{R}^{|V|}$ 分别对应输出投影矩阵和偏置项，而 d_h 则是RNN隐藏状态的大小。这些 $|V|$ 维度的 y_t 模型用于描述下一个单词的离散概率分布。

需要注意的是，上述公式并未对循环神经网络单元的具体结构做任何假设，f 它可以用标准的循环单元、门控循环单元（GRU）（Cho等人，2014年）、长短期记忆网络（LSTM）单元（Hochreiter与Schmidhuber，1997年）等来替代。在我们的实验中，我们使用了具有两层结构的LSTM单元。

Give在 t^{th} 示例中，会为该示例计算相应的损失值。RNNLMs中所使用的损失函数即为该损失值。
AI即为增强损失机制。
仅考虑 y_t 与观测到的独热词标记 y_t^*之间的交叉熵。

$$J_t = \text{CE}(y_t^* \parallel y_t) = -\sum_{i \in |V|} y_{t,i}^* \log y_{t,i}. \tag{2.4}$$

我们将把 y_t 称为 t^{th} 示例的模型预测分布，把 y_t^* 称为经验目标分布（实际上二者都是在给定历史信息条件下的条件分布）。由于当目标分布为独热形式时，交叉熵与库尔贝克-莱布勒散度是等价的，因此我们可以将 t^{th} 示例的损失函数重写为

$$J_t = \text{D}_{KL}(y_t^* \parallel y_t). \tag{2.5}$$

因此，我们可以将RNNLM中传统损失的优化视为试图最小化[1] 模型预测分布（y）与经验目标分布（y^*）之间的距离，而当训练样本数量充足时，这种优化方式就能逐渐接近于最小化与真实目标分布的距离。在我们即将介绍的框架中，由于库尔贝克-莱布勒散度能够直观地被理解为分布之间的距离，尽管在我们的框架中它与交叉熵并不等价，但我们仍然选择使用它而非交叉熵。

3 增强交叉熵损失机制

我们提议通过引入额外的损失项来对传统的交叉熵损失机制进行增强，具体方式如下：

$$\hat{y}_t = \text{softmax}\left(Wh_t/\tau\right), \tag{3.1}$$
$$J_t^{aug} = \text{D}_{KL}(\tilde{y}_t \parallel \hat{y}_t), \tag{3.2}$$
$$J_t^{tot} = J_t + \alpha J_t^{aug}. \tag{3.3}$$

在上述内容中，α 是需调整的超参数，而$\hat{y}_t$ 与常规模型预测分布 y_t 几乎完全相同，唯一的不同在于其逻辑值会被除以一个温度参数 τ。我们将 $\tilde{y}_t$ 定义为一种概率分布，用于在考虑词序历史的前提下估计真实数据分布，并且该分布需满足 $\mathbb{E}\tilde{y}_t = \mathbb{E}y_t^*$这一条件。该框架的目标便是最小化预测分布与对真实数据分布更精确的估计之间的分布差异。

[1] 不过需要指出的是，库尔贝克-莱布勒散度并非一种有效的距离度量。

distribution distance between the prediction distribution and a more accurate estimate of the true data distribution.

To understand the effect of optimizing in this setting, let's focus on an ideal case in which we are given the true data distribution so that $\tilde{y}_t = \mathbb{E}y_t^*$, and we only use the augmented loss, J^{aug}. We will carry out our investigation through stochastic gradient descent, which is the technique dominantly used for training neural networks. The gradient of J_t^{aug} with respect to the logits Wh_t is

$$\nabla J_t^{aug} = \frac{1}{\tau}(\hat{y}_t - \tilde{y}_t). \tag{3.4}$$

Let's denote by $e_j \in \mathbb{R}^{|V|}$ the vector whose j^{th} entry is 1, and others are zero. We can then rewrite (3.4) as

$$\tau \nabla J_t^{aug} = \hat{y}_t - \left[e_1, \ldots, e_{|V|}\right] \tilde{y}_t = \sum_{i \in V} \tilde{y}_{t,i}(\hat{y}_t - e_i). \tag{3.5}$$

Implication of (3.5) is the following: Every time the optimizer sees one training example, it takes a step not only on account of the label seen, but it proceeds taking into account all the class labels for which the conditional probability is not zero, and the relative step size for each step is given by the conditional probability for that label, $\tilde{y}_{t,i}$. Furthermore, this is a much less noisy update since the target distribution is exact and deterministic. Therefore, unless all the examples exclusively belong to a specific class with probability 1, the optimization will act much differently and train with greatly improved supervision.

The idea proposed in the recent work by Hinton et al. (2015) might be considered as an application of this framework, where they try to obtain a good set of $\tilde{y}$'s by training very large models and using the model prediction distributions of those.

Although finding a good $\tilde{y}$ in general is rather nontrivial, in the context of language modeling we can hope to achieve this by exploiting the inherent metric space of classes encoded into the model, namely the space of word embeddings. Specifically, we propose the following for $\tilde{y}$:

$$u_t = Ly_t^*, \tag{3.6}$$

$$\tilde{y}_t = \text{softmax}\left(\frac{L^T u_t}{\tau}\right). \tag{3.7}$$

In words, we first find the target word vector which corresponds to the target word token (resulting in u_t), and then take the inner product of the target word vector with all the other word vectors to get an unnormalized probability distribution. We adjust this with the same temperature parameter τ used for obtaining $\hat{y}_t$ and apply softmax. The target distribution estimate, $\tilde{y}$, therefore measures the similarity between the word vectors and assigns similar probability masses to words that the language model deems close. Note that the estimation of $\tilde{y}$ with this procedure is iterative, and the estimates of $\tilde{y}$ in the initial phase of the training are not necessarily informative. However, as training procedes, we expect $\tilde{y}$ to capture the word statistics better and yield a consistently more accurate estimate of the true data distribution.

4 Theoretically Driven Reuse of Word Embeddings

We now theoretically motivate and introduce a second modification to improve learning in the language model. We do this by analyzing the proposed augmented loss in a particular setting, and observe an implicit core mechanism of this loss. We then make our proposition by making this mechanism explicit.

We start by introducing our setting for the analysis. We restrict our attention to the case where the input embedding dimension is equal to the dimension of the RNN hidden state, i.e. $d \triangleq d_x = d_h$. We also set $b = 0$ in (2.3) so that $y_t = Wh_t$. We only use the augmented loss, i.e. $J^{tot} = J^{aug}$, and we assume that we can achieve zero training loss. Finally, we set the temperature parameter τ to be large.

We first show that when the temperature parameter, τ, is high enough, J_t^{aug} acts to match the logits of the prediction distribution to the logits of the the more informative labels, $\tilde{y}$. We proceed in the same

即预测分布与真实数据分布的更准确估计值之间的分布距离。

为了解析在此设定下进行优化所带来的影响，让我们聚焦于一种理想情况：此时我们掌握了真实的数据分布，即 $\tilde{y}_t = \mathbb{E}y_t^*$，且仅使用增强损失 J^{aug}。我们将通过随机梯度下降法来进行研究，这是用于训练神经网络的最主流技术。 J_t^{aug} 关于逻辑值 Wh_t 的梯度为

$$\nabla J_t^{aug} = \frac{1}{\tau}(\hat{y}_t - \tilde{y}_t). \tag{3.4}$$

我们用 $e_j \in \mathbb{R}^{|V|}$ 来表示这样一个向量：其 j^{th} 第i个分量为1，其余分量均为0。这样我们就可以将（3.4）重新表述为

$$\tau \nabla J_t^{aug} = \hat{y}_t - \left[e_1, \ldots, e_{|V|}\right] \tilde{y}_t = \sum_{i \in V} \tilde{y}_{t,i}(\hat{y}_t - e_i). \tag{3.5}$$

(3.5)所蕴含的含义如下：每次优化器处理一个训练样本时，它不仅会根据该样本的标签来决定更新步长，还会考虑所有条件概率不为零的类别标签，且每一步的相对步长由对应标签的条件概率决定，即$\tilde{y}_{t,\ i}$。此外，由于目标分布是精确且确定的，因此这种更新方式的抗干扰能力要强得多。所以，除非所有样本都以1的概率完全属于某个特定类别，否则优化过程的表现将会大不相同，模型的训练效果也会得到显著提升。

在Hinton等人（2015年）近期发表的论文中提出的理念，可被视为该框架的一种应用——他们试图通过训练规模极大的模型，并利用这些模型的预测分布，来获得一组优质的 $\tilde{y}$词嵌入矩阵。

尽管通常而言找到一个优秀的 $\tilde{y}$ 并非易事，但在语言建模的背景下，我们有望通过利用编码在模型中的类别固有度量空间——也就是词嵌入空间——来实现这一目标。具体而言，针对 $\tilde{y}$，我们提出了以下方法：

$$u_t = Ly_t^*, \tag{3.6}$$

$$\tilde{y}_t = \text{softmax}\left(\frac{L^T u_t}{\tau}\right). \tag{3.7}$$

简而言之，我们首先找到与目标词标记对应的目标词向量（即 u_t），然后计算该目标词向量与所有其他词向量的内积，从而得到一个未标准化的概率分布。接着，我们使用与获取$\hat{}y_t$ 时相同的温度参数τ 来调整这个分布，并应用softmax函数。如此得到的目标分布估计值 $\tilde{y}$便可用于衡量各词向量之间的相似性，同时为语言模型认为相近的词语分配相似的概率值。需要注意的是，通过这一流程对 $\tilde{y}$ 的估算属于迭代过程，在训练初期得到的 $\tilde{y}$ 估计值未必具有参考价值。但随着训练的进行，我们预计 $\tilde{y}$ 能够更好地捕捉词频统计特征，进而给出更为准确的真实数据分布估计值。

4 基于理论指导的词嵌入复用方法

接下来，我们从理论层面阐述并引入第二项改进措施，以此提升语言模型的学习能力。我们通过在特定场景下分析所提出的增强损失，发现了该损失背后隐含的核心机制；随后，我们通过将这一机制明确化，从而提出了相应的设计方案。

我们首先介绍用于分析的设定。此处我们将研究范围限制在输入嵌入维度等于RNN隐藏状态维度的情况，即 d ， $d_x = d_h$。同时我们还将 $b = 0$ 设为（2.3），从而使得 $y_t = Wh_t$。我们仅使用增强损失，也就是 $J^{tot} = J^{aug}$，并假设能够实现零训练损失。最后，我们将温度参数 τ 设置得较大。

我们首先证明：当温度参数 τ的值足够高时， J_t^{aug} 会促使预测分布的逻辑值与那些信息量更高的标签的逻辑值相匹配，即 $\tilde{y}$。后续的分析方法也与此一致。

way as was done in Hinton et al. (2015) to make an identical argument. Particularly, we consider the derivative of J_t^{aug} with respect to the entries of the logits produced by the neural network.

Let's denote by l_i the ith column of L. Using the first order approximation of exponential function around zero ($\exp(x) \approx 1 + x$), we can approximate $\tilde{y}_t$ (same holds for $\hat{y}_t$) at high temperatures as follows:

$$\tilde{y}_{t,i} = \frac{\exp\left(\langle u_t, l_i\rangle/\tau\right)}{\sum_{j\in V}\exp\left(\langle u_t, l_j\rangle/\tau\right)} \approx \frac{1 + \langle u_t, l_i\rangle/\tau}{|V| + \sum_{j\in V}\langle u_t, l_j\rangle/\tau}. \tag{4.1}$$

We can further simplify (4.1) if we assume that $\langle u_t, l_j\rangle = 0$ on average:

$$\tilde{y}_{t,i} \approx \frac{1 + \langle u_t, l_i\rangle/\tau}{|V|}. \tag{4.2}$$

By replacing $\tilde{y}_t$ and $\hat{y}_t$ in (3.4) with their simplified forms according to (4.2), we get

$$\frac{\partial J_t^{aug}}{\partial\left(Wh_t\right)_i} \to \frac{1}{\tau^2|V|}\left(Wh_t - L^T u_t\right)_i \quad \text{as } \tau \to \infty, \tag{4.3}$$

which is the desired result that augmented loss tries to match the logits of the model to the logits of $\tilde{y}$'s. Since the training loss is zero by assumption, we necessarily have

$$Wh_t = L^T u_t \tag{4.4}$$

for each training example, i.e., gradient contributed by each example is zero. Provided that W and L are full rank matrices and there are more linearly independent examples of h_t's than the embedding dimension d, we get that the space spanned by the columns of L^T is equivalent to that spanned by the columns of W. Let's now introduce a square matrix A such that $W = L^T A$. (We know A exists since L^T and W span the same column space). In this case, we can rewrite

$$Wh_t = L^T A h_t \triangleq L^T \tilde{h}_t. \tag{4.5}$$

In other words, by reusing the embedding matrix in the output projection layer (with a transpose) and letting the neural network do the necessary linear mapping $h \to Ah$, we get the same result as we would have in the first place.

Although the above scenario could be difficult to exactly replicate in practice, it uncovers a mechanism through which our proposed loss augmentation acts, which is trying to constrain the output (unnormalized) probability space to a small subspace governed by the embedding matrix. This suggests that we can make this mechanism explicit and constrain $W = L^T$ during training while setting the output bias, b, to zero. Doing so would not only eliminate a big matrix which dominates the network size for models with even moderately sized vocabularies, but it would also be optimal in our setting of loss augmentation as it would eliminate much work to be done by the augmented loss.

5 Related Work

Since their introduction in Mikolov et al. (2010), many improvements have been proposed for RNNLMs , including different dropout methods (Zaremba et al., 2014; Gal, 2015), novel recurrent units (Zilly et al., 2016), and use of pointer networks to complement the recurrent neural network (Merity et al., 2016). However, none of the improvements dealt with the loss structure, and to the best of our knowledge, our work is the first to offer a new loss framework.

Our technique is closely related to the one in Hinton et al. (2015), where they also try to estimate a more informed data distribution and augment the conventional loss with KL divergence between model prediction distribution and the estimated data distribution. However, they estimate their data distribution by training large networks on the data and then use it to improve learning in smaller networks. This is fundamentally different from our approach, where we improve learning by transferring knowledge between different parts of the same network, in a self contained manner.

The work we present in this paper is based on a report which was made public in Inan & Khosravi (2016). We have recently come across a concurrent preprint (Press & Wolf, 2016) where the authors reuse the word embedding matrix in the output projection to improve language modeling. However, their work is purely empirical, and they do not provide any theoretical justification for their

与 Hinton等人(2015年)所采用的方式相同，我们也能通过同样的方法得出相同的结论。具体而言，我们会研究 J_t^{aug} 对于神经网络生成的逻辑值各分量的导数。

我们用 l_i 来表示L的 i^{th} 列。利用指数函数在零点附近的一阶近似形式（$\exp(x) \approx 1 + x$），我们可以如下方式近似表示高温条件下的 $\tilde{y}_t$ （$\hat{y}_t$也同样适用）：

$$\tilde{y}_{t,i} = \frac{\exp\left(\langle u_t, l_i\rangle/\tau\right)}{\sum_{j\in V}\exp\left(\langle u_t, l_j\rangle/\tau\right)} \approx \frac{1 + \langle u_t, l_i\rangle/\tau}{|V| + \sum_{j\in V}\langle u_t, l_j\rangle/\tau}. \tag{4.1}$$

如果我们假设4.1），那么就可以进一步简化 $\langle u_t, l_j\rangle = 0$ 从平均意义上来看：

$$\tilde{y}_{t,i} \approx \frac{1 + \langle u_t, l_i\rangle/\tau}{|V|}. \tag{4.2}$$

通过将（3.4）中的和$\hat{y}_t$替换为根据（4.2）给出的简化形式，我们可以得到

$$\frac{\partial J_t^{aug}}{\partial\left(Wh_t\right)_i} \to \frac{1}{\tau^2|V|}\left(Wh_t - L^T u_t\right)_i \quad \text{as } \tau \to \infty, \tag{4.3}$$

这正是增强损失试图实现的预期结果——将模型的逻辑值与$\tilde{y}$的逻辑值对齐。由于按照假设训练损失为零，我们必然可以得出

$$Wh_t = L^T u_t \tag{4.4}$$

对于每一个训练样本而言，即每个样本贡献的梯度均为零。只要 W 和 L是满秩矩阵，且 h_t 中线性独立的样本数量多于嵌入维度 d，那么由 L^T 的列所张成的空间就等同于由 W的列所张成的空间。现在让我们引入一个方阵 A，使得 $W = L^T A$成立。（由于 L^T 和 W 张成了相同的列空间，因此我们知道 A 是存在的。）在这种情况下，我们可以重新表述为

$$Wh_t = L^T A h_t \triangleq L^T \tilde{h}_t. \tag{4.5}$$

换言之，通过在输出投影层中复用经过转置的词嵌入矩阵，并让神经网络执行必要的线性映射 $h \to Ah$，我们就能得到与最初相同的结果。

尽管上述场景在实践中很难被完全复现，但它揭示了我们所提出的增强损失机制的作用原理——该机制试图将输出（未归一化的）概率空间限制在由词嵌入矩阵控制的一个较小子空间内。这表明我们可以通过在设置输出偏置 b为零的同时，明确体现这一机制并在训练过程中对 $W = L^T$进行约束。这样做不仅能消除那些在词汇量仅为中等规模的模型中就会占据巨大网络规模的庞大矩阵，而且在我们这种采用增强损失的架构中也是最优选择，因为这样可以减少增强损失需要承担的大量计算工作。

5 相关工作

Since their introduction in Mikolov et al. (2010), many improvements have been proposed for RNNLMs , including different dropout methods (Zaremba et al., 2014; Gal, 2015), novel recurrent units (Zilly et al., 2016), and use of pointer networks to complement the recurrent neural network (Merity et al., 2016). However, none of the improvements dealt with the loss structure, and to the best of our knowledge, our work is the first to offer a new loss framework.

我们的技术与 Hinton等人（2015年）所提出的方法密切相关。在该研究中，他们同样试图估算更为准确的数据分布，并通过模型预测分布与估算得到的数据分布之间的KL散度来增强传统损失函数。不过，他们是通过在数据上训练大型模型来估算数据分布，再利用该结果提升小型模型的学习效果。而这与我们的方法有着本质区别——我们是通过在同一模型的不同部分之间传递知识，以一种自包含的方式来实现学习效果的提升。

The work we present in this paper is based on a report which was made public in Inan & Khosravi (2016). We have recently come across a concurrent preprint (Press & Wolf, 2016) where the authors reuse the word embedding matrix in the output projection to improve language modeling. However, their work is purely empirical, and they do not provide any theoretical justification for their

approach. Finally, we would like to note that the idea of using the same representation for input and output words has been explored in the past, and there exists language models which could be interpreted as simple neural networks with shared input and output embeddings (Bengio et al., 2001; Mnih & Hinton, 2007). However, shared input and output representations were implicitly built into these models, rather than proposed as a supplement to a baseline. Consequently, possibility of improvement was not particularly pursued by sharing input and output representations.

6 EXPERIMENTS

In our experiments, we use the Penn Treebank corpus (PTB) (Marcus et al., 1993), and the Wikitext-2 dataset (Merity et al., 2016). PTB has been a standard dataset used for benchmarking language models. It consists of 923k training, 73k validation, and 82k test words. The version of this dataset which we use is the one processed in Mikolov et al. (2010), with the most frequent 10k words selected to be in the vocabulary and rest replaced with a an <unk> token [2]. Wikitext-2 is a dataset released recently as an alternative to PTB[3]. It contains $2,088$k training, 217k validation, and 245k test tokens, and has a vocabulary of $33,278$ words; therefore, in comparison to PTB, it is roughly 2 times larger in dataset size, and 3 times larger in vocabulary.

6.1 MODEL AND TRAINING HIGHLIGHTS

We closely follow the LSTM based language model proposed in Zaremba et al. (2014) for constructing our baseline model. Specifically, we use a 2-layer LSTM with the same number of hidden units in each layer, and we use 3 different network sizes: small (200 units), medium (650 units), and large (1500 units). We train our models using stochastic gradient descent, and we use a variant of the dropout method proposed in Gal (2015). We defer further details regarding training the models to section A of the appendix. We refer to our baseline network as variational dropout LSTM, or VD-LSTM in short.

6.2 EMPIRICAL VALIDATION FOR THE THEORY OF REUSING WORD EMBEDDINGS

In Section 4, we showed that the particular loss augmentation scheme we choose constrains the output projection matrix to be close to the input embedding matrix, without explicitly doing so by reusing the input embedding matrix. As a first experiment, we set out to validate this theoretical result. To do this, we try to simulate the setting in Section 4 by doing the following: We select a randomly chosen $20,000$ contiguous word sequence in the PTB training set, and train a 2-layer LSTM language model with 300 units in each layer with loss augmentation by minimizing the following loss:

$$J^{tot} = \beta J^{aug} \tau^2 |V| + (1 - \beta) J. \tag{6.1}$$

Here, β is the proportion of the augmented loss used in the total loss, and J^{aug} is scaled by $\tau^2|V|$ to approximately match the magnitudes of the derivatives of J and J^{aug} (see (4.3)). Since we aim to achieve the minimum training loss possible, and the goal is to show a particular result rather than to achieve good generalization, we do not use any kind of regularization in the neural network (e.g. weight decay, dropout). For this set of experiments, we also constrain each row of the input embedding matrix to have a norm of 1 because training becomes difficult without this constraint when only augmented loss is used. After training, we compute a metric that measures distance between the subspace spanned by the rows of the input embedding matrix, L, and that spanned by the columns of the output projection matrix, W. For this, we use a common metric based on the relative residual norm from projection of one matrix onto another (Björck & Golub, 1973). The computed distance between the subspaces is 1 when they are orthogonal, and 0 when they are the same. Interested reader may refer to section B in the appendix for the details of this metric.

Figure 1 shows the results from two tests. In one (panel a), we test the effect of using the augmented loss by sweeping β in (6.1) from 0 to 1 at a reasonably high temperature ($\tau = 10$). With no loss augmentation ($\beta = 0$), the distance is almost 1, and as more and more augmented loss is used the

[2]PTB can be downloaded at http://www.fit.vutbr.cz/ imikolov/rnnlm/simple-examples.tgz

[3]Wikitext-2 can be downloaded at https://s3.amazonaws.com/research.metamind.io/wikitext/wikitext-2-v1.zip

approach. Finally, we would like to note that the idea of using the same representation for input and output words has been explored in the past, and there exists language models which could be interpreted as simple neural networks with shared input and output embeddings (Bengio et al., 2001; Mnih & Hinton, 2007). However, shared input and output representations were implicitly built into these models, rather than proposed as a supplement to a baseline. Consequently, possibility of improvement was not particularly pursued by sharing input and output representations.

6项实验

In our experiments, we use the Penn Treebank corpus (PTB) (Marcus et al., 1993), and the Wikitext-2 dataset (Merity et al., 2016). PTB has been a standard dataset used for benchmarking language models. It consists of 923k training, 73k validation, and 82k test words. The version of this dataset which we use is the one processed in Mikolov et al. (2010), with the most frequent 10k words selected to be in the vocabulary and rest replaced with a an <unk> token [2]. Wikitext-2 is a dataset released recently as an alternative to PTB[3]. It contains $2,088$k training, 217k validation, and 245k test tokens, and has a vocabulary of $33,278$ words; therefore, in comparison to PTB, it is roughly 2 times larger in dataset size, and 3 times larger in vocabulary.

6.1 模型的训练要点

We closely follow the LSTM based language model proposed in Zaremba et al. (2014) for constructing our baseline model. Specifically, we use a 2-layer LSTM with the same number of hidden units in each layer, and we use 3 different network sizes: small (200 units), medium (650 units), and large (1500 units). We train our models using stochastic gradient descent, and we use a variant of the dropout method proposed in Gal (2015). We defer further details regarding training the models to section A of the appendix. We refer to our baseline network as variational dropout LSTM, or VD-LSTM in short.

6.2 词嵌入复用理论的实证验证

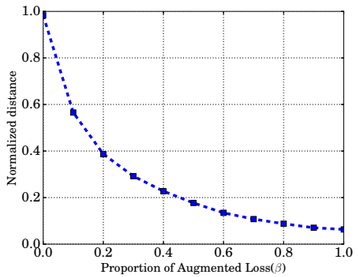
In Section 4, we showed that the particular loss augmentation scheme we choose constrains the output projection matrix to be close to the input embedding matrix, without explicitly doing so by reusing the input embedding matrix. As a first experiment, we set out to validate this theoretical result. To do this, we try to simulate the setting in Section 4 by doing the following: We select a randomly chosen $20,000$ contiguous word sequence in the PTB training set, and train a 2-layer LSTM language model with 300 units in each layer with loss augmentation by minimizing the following loss:

$$J^{tot} = \beta J^{aug} \tau^2 |V| + (1 - \beta) J. \tag{6.1}$$

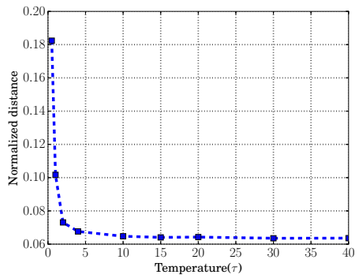
Here, β is the proportion of the augmented loss used in the total loss, and J^{aug} is scaled by $\tau^2|V|$ to approximately match the magnitudes of the derivatives of J and J^{aug} (see (4.3)). Since we aim to achieve the minimum training loss possible, and the goal is to show a particular result rather than to achieve good generalization, we do not use any kind of regularization in the neural network (e.g. weight decay, dropout). For this set of experiments, we also constrain each row of the input embedding matrix to have a norm of 1 because training becomes difficult without this constraint when only augmented loss is used. After training, we compute a metric that measures distance between the subspace spanned by the rows of the input embedding matrix, L, and that spanned by the columns of the output projection matrix, W. For this, we use a common metric based on the relative residual norm from projection of one matrix onto another (Björck & Golub, 1973). The computed distance between the subspaces is 1 when they are orthogonal, and 0 when they are the same. Interested reader may refer to section B in the appendix for the details of this metric.

图 1 展示了两项测试的结果。在其中一项测试（面板a）中，我们在相当高的温度（$\tau = 10$）下，通过将 β 在（6.1）范围内从0 逐渐变化到1，来研究使用增强损失的效果。在没有应用增强损失（$\beta = 0$）的情况下，该距离几乎为<style '30'>1，而随着越来越多地使用增强损失，这一数值会

[2]PTB数据集可在此处下载：http://www.fit.vutbr.cz/ imikolov/rnnlm/simple-examples.tgz [3]
Wikitext-2数据集则可在此处获取：https://s3.amazonaws.com/research.metamind.io/wikitext/wikitext-2-v1.zip



(a) Subspace distance at $\tau = 10$ for different proportions of J^{aug}



(b) Subspace distance at different temperatures when only J^{aug} is used

Figure 1: Subspace distance between L^T and W for different experiment conditions for the validation experiments. Results are averaged over 10 independent runs. These results validate our theory under practical conditions.

distance decreases rapidly, and eventually reaches around 0.06 when only augmented loss is used. In the second test (panel b), we set $\beta = 1$, and try to see the effect of the temperature on the subspace distance (remember the theory predicts low distance when $\tau \to \infty$). Notably, the augmented loss causes W to approach L^T sufficiently even at temperatures as low as 2, although higher temperatures still lead to smaller subspace distances.

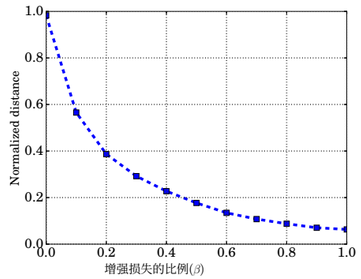
These results confirm the mechanism through which our proposed loss pushes W to learn the same column space as L^T, and it suggests that reusing the input embedding matrix by explicitly constraining $W = L^T$ is not simply a kind of regularization, but is in fact an optimal choice in our framework. What can be achieved separately with each of the two proposed improvements as well as with the two of them combined is a question of empirical nature, which we investigate in the next section.

6.3 Results on PTB and Wikitext-2 Datasets

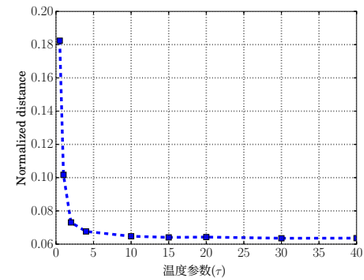
In order to investigate the extent to which each of our proposed improvements helps with learning, we train 4 different models for each network size: (1) 2-Layer LSTM with variational dropout (VD-LSTM) (2) 2-Layer LSTM with variational dropout and augmented loss (VD-LSTM +AL) (3) 2-Layer LSTM with variational dropout and reused embeddings (VD-LSTM +RE) (4) 2-Layer LSTM with variational dropout and both RE and AL (VD-LSTM +REAL).

Figure 2 shows the validation perplexities of the four models during training on the PTB corpus for small (panel a) and large (panel b) networks. All of AL, RE, and REAL networks significantly outperform the baseline in both cases. Table 1 compares the final validation and test perplexities of the four models on both PTB and Wikitext-2 for each network size. In both datasets, both AL and RE improve upon the baseline individually, and using RE and AL together leads to the best performance. Based on performance comparisons, we make the following notes on the two proposed improvements:

- AL provides better performance gains for smaller networks. This is not surprising given the fact that small models are rather inflexible, and one would expect to see improved learning by training against a more informative data distribution (contributed by the augmented loss) (see Hinton et al. (2015)). For the smaller PTB dataset, performance with AL surpasses that with RE. In comparison, for the larger Wikitext-2 dataset, improvement by AL is more limited. This is expected given larger training sets better represent the true data distribution, mitigating the supervision problem. In fact, we set out to validate this reasoning in a direct manner, and additionally train the small networks separately on the first and second halves of the Wikitext-2 training set. This results in two distinct datasets which are each about the same size as PTB (1044K vs 929K). As can be seen in Table 2, AL



(a) 当 $\tau = 10$ 时对应的子空间距离在不同比例的 J^{aug}情况下



(b) 在不同温度参数下的子空间距离仅使用 J^{aug} 时的表现

图1：验证实验中不同实验条件下 L^T 与 W 的子空间距离。数值为10次独立实验次数的平均值。这些结果证明了我们的理论在实际应用环境中的有效性。

子空间距离会迅速减小，当仅使用增强损失时最终会降至约0.06左右。在第二个测试（图b）中，我们设置了 $\beta = 1$，并试图观察温度参数对子空间距离的影响（请记住，根据理论，当 $\tau \to \infty$时距离应当较低）。值得注意的是，即便在低至2的温度下，增强损失也能促使 W充分接近 L^T，不过更高的温度依然能进一步缩小子空间距离。

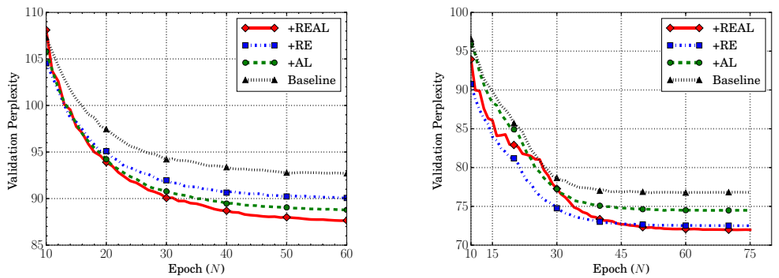
这些结果验证了我们所提出的损失函数促使 W 学习与 L^T相同的列空间所依赖的机制，同时表明，通过明确约束 $W = L^T$ 来复用输入嵌入矩阵并非单纯的正则化手段，实际上在我们的框架中才是最优选择。单独应用这两种提出的改进方法，或是将它们结合起来使用能够实现何种效果，属于经验性研究范畴，我们将在下一节中进行探讨。

6.3 PTB数据集与WIKIText-2数据集上的实验结果

为探究我们所提出的各项改进在提升学习效果方面的作用程度，我们对每种网络规模训练了4种不同的模型：(1) 采用变分丢弃机制的2层长短期记忆网络（VD-LSTM）；(2) 同时应用变分丢弃机制与增强损失的2层长短期记忆网络（VD-LSTM +AL）；(3) 采用变分丢弃机制并复用嵌入矩阵的2层长短期记忆网络（VD-LSTM +RE）；(4) 同时运用丢弃机制、嵌入矩阵复用机制以及增强损失的2层长短期记忆网络（VD-LSTM +REAL）。

图 2 展示了这四种模型在PTB语料库上训练时，针对小型网络（面板a）和大型网络（面板b）的验证困惑度情况。在这两种场景下，所有的增强损失机制、嵌入矩阵复用机制以及REAL模块均显著优于基线模型。表 1 则列出了不同网络规模下，这四种模型在PTB数据集和Wikitext-2数据集上的最终验证困惑度与测试困惑度对比结果。在两个数据集中，增强损失机制和嵌入矩阵复用机制都能单独提升模型的性能，而将二者结合使用则能带来最佳效果。基于上述性能对比，我们对这两种提出的改进方案总结如下：

- AL provides better performance gains for smaller networks. This is not surprising given the fact that small models are rather inflexible, and one would expect to see improved learning by training against a more informative data distribution (contributed by the augmented loss) (see Hinton et al. (2015)). For the smaller PTB dataset, performance with AL surpasses that with RE. In comparison, for the larger Wikitext-2 dataset, improvement by AL is more limited. This is expected given larger training sets better represent the true data distribution, mitigating the supervision problem. In fact, we set out to validate this reasoning in a direct manner, and additionally train the small networks separately on the first and second halves of the Wikitext-2 training set. This results in two distinct datasets which are each about the same size as PTB (1044K vs 929K). As can be seen in Table 2, AL



(a) Small Network

(b) Large Network

Figure 2: Progress of validation perplexities during training for the 4 different models for two (small (200) and large (1500)) network sizes.

has significantly improved competitive performance against RE and REAL despite the fact that embedding size is 3 times larger compared to PTB. These results support our argument that the proposed augmented loss term acts to improve the amount of information gathered from the dataset.

- RE significantly outperforms AL for larger networks. This indicates that, for large models, the more effective mechanism of our proposed framework is the one which enforces proximity between the output projection space and the input embedding space. From a model complexity perspective, the nontrivial gains offered by RE for all network sizes and for both datasets could be largely attributed to its explicit function to reduce the model size while preserving the representational power according to our framework.

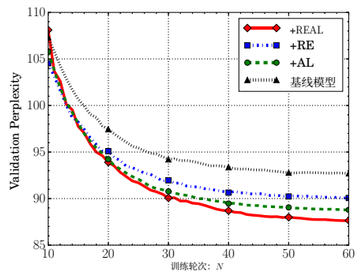
We list in Table 3 the comparison of models with and without our proposed modifications on the Penn Treebank Corpus. The best LSTM model (VD-LSTM+REAL) outperforms all previous work which uses conventional framework, including large ensembles. The recently proposed recurrent highway networks (Zilly et al., 2016) when trained with reused embeddings (VD-RHN +RE) achieves the best overall performance, improving on VD-RHN by a perplexity of 2.5.

Table 1: Comparison of the final word level perplexities on the validation and test set for the 4 different models.

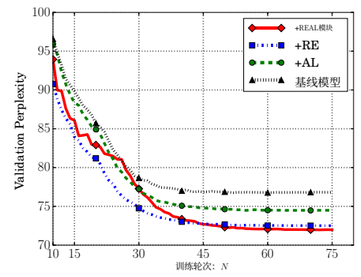
		PTB		Wikitext-2	
Network	**Model**	**Valid**	**Test**	**Valid**	**Test**
Small[4] (200 units)	VD-LSTM	92.6	87.3	112.2	105.9
	VD-LSTM+AL	86.3	82.9	110.3	103.8
	VD-LSTM+RE	89.9	85.1	106.1	100.5
	VD-LSTM+REAL	86.3	82.7	105.6	98.9
Medium (650 units)	VD-LSTM	82.0	77.7	100.2	95.3
	VD-LSTM+AL	77.4	74.7	98.8	93.1
	VD-LSTM+RE	77.1	73.9	92.3	87.7
	VD-LSTM+REAL	75.7	73.2	91.5	87.0
Large[5] (1500 units)	VD-LSTM	76.8	72.6	-	-
	VD-LSTM+AL	74.5	71.2	-	-
	VD-LSTM+RE	72.5	69.0	-	-
	VD-LSTM+REAL	71.1	68.5	-	-

[4]For PTB, small models were re-trained by initializing to their final configuration from the first training session. This did not change the final perplexity for baseline, but lead to improvements for the other models.

[5]Large network results on Wikitext-2 are not reported since computational resources were insufficient to run some of the configurations.



(a) 小型网络



(b) 大规模网络

图2：针对两种不同网络规模（小型（200）和大型（1500）），4种不同模型在训练过程中验证困惑度的变化趋势。

尽管其嵌入维度是PTB的3倍，该模型在面对RE和REAL时的竞争性能仍得到了显著提升。这些结果为我们的观点提供了支撑，即所提出的增强损失项能够有效增加从数据集中获取的信息量。

在规模较大的模型中，● 嵌入矩阵复用机制的性能显著优于增强损失机制。这表明，对于大型模型而言，我们提出的框架中更为有效的机制正是那种能够使输出投影空间与输入嵌入空间保持相近性的机制。从模型复杂度的角度来看，无论网络规模大小或数据集类型如何，嵌入矩阵复用机制所带来的显著优势，在很大程度上都归功于它根据我们的框架设计，能够在保留表示能力的同时有效降低模型规模的功能。

We list in Table 3 the comparison of models with and without our proposed modifications on the Penn Treebank Corpus. The best LSTM model (VD-LSTM+REAL) outperforms all previous work which uses conventional framework, including large ensembles. The recently proposed recurrent highway networks (Zilly et al., 2016) when trained with reused embeddings (VD-RHN +RE) achieves the best overall performance, improving on VD-RHN by a perplexity of 2.5.

表1：4种不同模型在验证集和测试集上的最终词级困惑度对比。

		PTB		Wikitext-2数据集	
网络	模型	有效	**Test**	有效	**Test**
小型[4] (200 单元数)	VD-LSTM	92.6	87.3	112.2	105.9
	VD-LSTM+增强损失机制	86.3	82.9	110.3	103.8
	VD-LSTM+嵌入矩阵复用机制	89.9	85.1	106.1	100.5
	VD-LSTM+REAL模块	86.3	82.7	105.6	98.9
中等规模 (650 单元数)	VD-LSTM	82.0	77.7	100.2	95.3
	VD-LSTM+增强损失机制	77.4	74.7	98.8	93.1
	VD-LSTM+嵌入矩阵复用机制	77.1	73.9	92.3	87.7
	VD-LSTM+REAL模块	75.7	73.2	91.5	87.0
大型[5] (1500 单元数)	VD-LSTM	76.8	72.6	-	-
	VD-LSTM+增强损失机制	74.5	71.2	-	-
	VD-LSTM+嵌入矩阵复用机制	72.5	69.0	-	-
	VD-LSTM+REAL模块	71.1	68.5	-	-

[4]针对PTB数据集，小型模型是通过采用第一次训练环节得到的最终配置进行重新训练的。这样一来，基线模型的最终困惑度并未发生变化，而其他模型的性能则得到了提升。[5]由于计算资源不足，无法运行部分配置，因此没有公布大型模型在Wikitext-2数据集上的测试结果。

Table 2: Performance of the four different small models trained on the equally sized two partitions of Wikitext-2 training set. These results are consistent with those on PTB (see Table 1), which has a similar training set size with each of these partitions, although its word embedding dimension is three times smaller.

		Wikitext-2, Partition 1		**Wikitext-2, Partition 2**	
Network	**Model**	**Valid**	**Test**	**Valid**	**Test**
Small (200 units)	VD-LSTM	159.1	148.0	163.19	148.6
	VD-LSTM+AL	153.0	142.5	156.4	143.7
	VD-LSTM+RE	152.4	141.9	152.5	140.9
	VD-LSTM+REAL	149.3	140.6	150.5	138.4

Table 3: Comparison of our work to previous state of the art on word-level validation and test perplexities on the Penn Treebank corpus. Models using our framework significantly outperform other models.

Model	**Parameters**	**Validation**	**Test**
RNN (Mikolov & Zweig)	6M	-	124.7
RNN+LDA (Mikolov & Zweig)	7M	-	113.7
RNN+LDA+KN-5+Cache (Mikolov & Zweig)	9M	-	92.0
Deep RNN (Pascanu et al., 2013a)	6M	-	107.5
Sum-Prod Net (Cheng et al., 2014)	5M	-	100.0
LSTM (medium) (Zaremba et al., 2014)	20M	86.2	82.7
CharCNN (Kim et al., 2015)	19M	-	78.9
LSTM (large) (Zaremba et al., 2014)	66M	82.2	78.4
VD-LSTM (large, untied, MC) (Gal, 2015)	66M	-	73.4 $\pm$ 0.0
Pointer Sentinel-LSTM(medium) (Merity et al., 2016)	21M	72.4	70.9
38 Large LSTMs (Zaremba et al., 2014)	2.51B	71.9	68.7
10 Large VD-LSTMs (Gal, 2015)	660M	-	68.7
VD-RHN (Zilly et al., 2016)	32M	71.2	68.5
VD-LSTM +REAL (large)	51M	71.1	68.5
VD-RHN +RE (Zilly et al., 2016) [6]	24M	**68.1**	**66.0**

6.4 Qualitative Results

One important feature of our framework that leads to better word predictions is the explicit mechanism to assign probabilities to words not merely according to the observed output statistics, but also considering the metric similarity between words. We observe direct consequences of this mechanism qualitatively in the Penn Treebank in different ways: First, we notice that the probability of generating the <unk> token with our proposed network (VD-LSTM +REAL) is significantly lower compared to the baseline network (VD-LSTM) across many words. This could be explained by noting the fact that the <unk> token is an aggregated token rather than a specific word, and it is often not expected to be close to specific words in the word embedding space. We observe the same behavior with very frequent words such as "a", "an", and "the", owing to the same fact that they are not correlated with particular words. Second, we not only observe better probability assignments for the target words, but we also observe relatively higher probability weights associated with the words close to the targets. Sometimes this happens in the form of predicting words semantically close together which are plausible even when the target word is not successfully captured by the model. We provide a few examples from the PTB test set which compare the prediction performance of 1500 unit VD-LSTM and 1500 unit VD-LSTM +REAL in table 4. We would like to note that prediction performance of VD-LSTM +RE is similar to VD-LSTM +REAL for the large network.

[6]This model was developed following our work in Inan & Khosravi (2016).

表2：在Wikitext-2训练集的两个规模相等的划分区上训练的四种不同小型模型的性能表现。这些结果与PTB数据集上的测试结果一致（见表1），因为PTB的每个划分区也具有类似的规模，只不过其词嵌入维数仅为前者的三分之一。

		Wikitext-2，第一划分区		Wikitext-2，第二划分区	
网络	模型	有效	**Test**	有效	**Test**
小型 (200 单元数)	VD-LSTM网络	159.1	148.0	163.19	148.6
	VD-LSTM网络+增强损失机制	153.0	142.5	156.4	143.7
	VD-LSTM网络+嵌入矩阵复用机制	152.4	141.9	152.5	140.9
	VD-LSTM网络+REAL模块	149.3	140.6	150.5	138.4

表3：我们在Penn Treebank语料库上的词级验证与测试困惑度指标方面，与现有最先进方法的对比情况。采用我们所提框架的模型性能显著优于其他模型。

模型	参数	验证	**Test**
RNN（米科洛夫与茨威格）	6M	-	124.7
RNN+LDA模型（米科洛夫与茨威格）	7M	-	113.7
RNN+LDA模型+KN-5+缓存结构（米科洛夫与茨威格）	9M	-	92.0
深度循环神经网络（Pascanu等人，2013a）	6M	-	107.5
和积网络（Cheng等人。，2014年）	5M	-	100.0
长短期记忆网络（中等规模）（扎伦巴等人，2014年）	20M	86.2	82.7
字符卷积网络（金等人，2015年）	19M	-	78.9
长短期记忆网络（大型）（扎伦巴等人，2014年）	66M	82.2	78.4
VD-LSTM（大型、未关联、多类别）(Gal，2015年)	66M	-	73.4 $\pm$ 0.0
指针哨兵长短期记忆网络（中等规模）（Merity等人，2016年）	21M	72.4	70.9
38个大型LSTM模型（扎伦巴等人，2014年）	25.1亿	71.9	68.7
10个大型VD-LSTM模型（Gal，2015年）	660M	-	68.7
VD-RHN（Zilly等人，2016年）	32M	71.2	68.5
VD-LSTM +REAL（大型）	51M	71.1	68.5
VD-RHN +RE (Zilly等人，2016年) [6]	24M	**68.1**	**66.0**

6.4 定性结果

One important feature of our framework that leads to better word predictions is the explicit mechanism to assign probabilities to words not merely according to the observed output statistics, but also considering the metric similarity between words. We observe direct consequences of this mechanism qualitatively in the Penn Treebank in different ways: First, we notice that the probability of generating the <unk> token with our proposed network (VD-LSTM +REAL) is significantly lower compared to the baseline network (VD-LSTM) across many words. This could be explained by noting the fact that the <unk> token is an aggregated token rather than a specific word, and it is often not expected to be close to specific words in the word embedding space. We observe the same behavior with very frequent words such as "a", "an", and "the", owing to the same fact that they are not correlated with particular words. Second, we not only observe better probability assignments for the target words, but we also observe relatively higher probability weights associated with the words close to the targets. Sometimes this happens in the form of predicting words semantically close together which are plausible even when the target word is not successfully captured by the model. We provide a few examples from the PTB test set which compare the prediction performance of 1500 unit VD-LSTM and 1500 unit VD-LSTM +REAL in table 4. We would like to note that prediction performance of VD-LSTM +RE is similar to VD-LSTM +REAL for the large network.

[6]该模型是在我们基于Inan与Khosravi（2016年）的研究成果基础上开发而成的

Table 4: Prediction for the next word by the baseline (VD-LSTM) and proposed (VD-LSTM +REAL) networks for a few example phrases in the PTB test set. Top 10 word predictions are sorted in descending probability, and are arranged in column-major format.

Phrase + *Next word(s)*	**Top 10 predicted words** VD-LSTM		**Top 10 predicted words** VD-LSTM +REAL	
information international said it believes that the complaints filed in + *federal court*	the 0.27	an 0.03	**federal 0.22**	connection 0.03
	a 0.13	august 0.01	the 0.1	august 0.03
	federal 0.13	new 0.01	a 0.08	july 0.03
	N 0.09	response 0.01	N 0.06	an 0.03
	⟨unk⟩ 0.05	connection 0.01	state 0.04	september 0.03
oil company refineries ran flat out to prepare for a robust holiday driving season in july and + *august*	the 0.09	in 0.03	**august 0.08**	a 0.03
	N 0.08	has 0.03	N 0.05	in 0.03
	a 0.07	is 0.02	early 0.05	that 0.02
	⟨unk⟩ 0.07	will 0.02	september 0.05	ended 0.02
	was 0.04	its 0.02	the 0.03	its 0.02
southmark said it plans to ⟨unk⟩ its ⟨unk⟩ to provide financial results as soon as its audit is + *completed*	the 0.06	to 0.03	expected 0.1	a 0.03
	⟨unk⟩ 0.05	likely 0.03	**completed 0.04**	scheduled 0.03
	a 0.05	expected 0.03	⟨unk⟩ 0.03	n't 0.03
	in 0.04	scheduled 0.01	the 0.03	due 0.02
	n't 0.04	**completed 0.01**	in 0.03	to 0.01
merieux said the government 's minister of industry science and + *technology*	⟨unk⟩ 0.33	industry 0.01	⟨unk⟩ 0.09	industry 0.03
	the 0.06	commerce 0.01	health 0.08	business 0.02
	a 0.01	planning 0.01	development 0.04	telecomm. 0.02
	other 0.01	management 0.01	the 0.04	human 0.02
	others 0.01	mail 0.01	a 0.03	other 0.01

7 CONCLUSION

In this work, we introduced a novel loss framework for language modeling. Particularly, we showed that the metric encoded into the space of word embeddings could be used to generate a more informed data distribution than the one-hot targets, and that additionally training against this distribution improves learning. We also showed theoretically that this approach lends itself to a second improvement, which is simply reusing the input embedding matrix in the output projection layer. This has an additional benefit of reducing the number of trainable variables in the model. We empirically validated the theoretical link, and verified that both proposed changes do in fact belong to the same framework. In our experiments on the Penn Treebank corpus and Wikitext-2, we showed that our framework outperforms the conventional one, and that even the simple modification of reusing the word embedding in the output projection layer is sufficient for large networks.

The improvements achieved by our framework are not unique to vanilla language modeling, and are readily applicable to other tasks which utilize language models such as neural machine translation, speech recognition, and text summarization. This could lead to significant improvements in such models especially with large vocabularies, with the additional benefit of greatly reducing the number of parameters to be trained.

表4：PTB测试集中若干示例短语的预测结果，展示了基线模型（VD-LSTM）与所提出的模型（VD-LSTM + REAL）对下一个词的预测情况。排名前10的候选词按出现概率从高到低排序，并以列优先格式呈现。

短语 + 下一个词	排名前10的预测词 VD-LSTM		排名前10的预测词 VD-LSTM +REAL	
国际信息 称其认为…… 在……提交的投诉 +联邦法院	0.27	0.03	联邦值：0.22	连接强度：0.03
	0.13	0.01	该数值为：0.1	august：0.03
	0.13	0.01	a：0.08	july：0.03
	0.09	响应值：0.01	N：0.06	an：0.03
	⟨unk⟩ 0.05	连接强度：0.01	state：0.04	9月：0.03
石油公司的炼油厂 全力投入准备工作中 为了度过一个安稳的假期 7月的驾驶高峰期以及…… + 8月	为0.09	在0.03范围内	august 0.08	a 0.03
	数值为0.08	包含0.03的数值	N：0.05	in：0.03
	值为0.07	为0.02	early：0.05	that：0.02
	⟨unk⟩ 0.07	will 0.02	september：0.05	ended：0.02
	was 0.04	its 0.02	0.03	0.02
southmark表示其有相关计划 要 ⟨unk⟩ 其 ⟨unk⟩ 到 提供财务相关数据 一旦其审计工作完成 + 已完成	0.06	降至0.03	预期值为 0.1	0.03
	⟨unk⟩ 0.05	大概为0.03	0.04	0.03
	0.05	预期值为 0.03	⟨unk⟩ 0.03	0.03
	实际值为 0.04	计划值为 0.01	0.03	0.02
	实际值为 0.04	已完成，值为 0.01	0.03	0.01
Merieux表示 政府的部长 工业、科学及 + 技术	⟨unk⟩ 0.33	工业领域：0.01	⟨unk⟩ 0.09	行业：0.03
	该类别：0.06	商业领域：0.01	健康领域：0.08	商业领域：0.02
	某一类别：0.01	规划：0.01	开发领域：0.04	远程通信领域：0.02
	其他：0.01	管理：0.01	“the”一词：0.04	人类 0.02
	其余类别：0.01	邮件：0.01	0.03	其他 0.01

7 结论

在本研究中，我们提出了一种用于语言建模的全新损失框架。具体而言，我们证明了编码在词嵌入空间中的度量值能够生成比独热编码目标更具信息量的数据分布，而且基于该分布进行训练还能进一步提升学习效果。我们还从理论上证明了这种方法还能带来第二重改进，即直接在输出投影层中复用输入嵌入矩阵；这样做还有一个额外的好处，那就是能减少模型中需要训练的变量数量。我们通过实验验证了这一理论关联，并确认所提出的这两种改进确实属于同一个框架。在基于Penn Treebank语料库和Wikitext-2数据集进行的实验中，我们证明了该框架的性能优于传统方法，即便只是简单地在输出投影层中复用词嵌入，对于大型网络而言也已足够有效。

我们的框架所带来的改进并非仅限于传统的语言建模领域，它同样可以轻松应用于其他需要使用语言模型的任务，比如神经机器翻译、语音识别以及文本摘要生成。这些应用场景中的模型有望因此获得显著提升，尤其是那些拥有庞大词汇量的模型；同时，该框架还能大幅减少需要训练的参数数量，带来额外的优势。

REFERENCES

Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. A neural probabilistic language model. 2001. URL http://www.iro.umontreal.ca/~lisa/pointeurs/nips00_lm.ps.

Åke Björck and Gene H Golub. Numerical methods for computing angles between linear subspaces. *Mathematics of computation*, 27(123):579–594, 1973.

Wei-Chen Cheng, Stanley Kok, Hoai Vu Pham, Hai Leong Chieu, and Kian Ming Adam Chai. Language modeling with sum-product networks. 2014.

Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.

Orhan Firat, Kyunghyun Cho, and Yoshua Bengio. Multi-way, multilingual neural machine translation with a shared attention mechanism. *arXiv preprint arXiv:1601.01073*, 2016.

Charlie Frogner, Chiyuan Zhang, Hossein Mobahi, Mauricio Araya, and Tomaso A Poggio. Learning with a wasserstein loss. In *Advances in Neural Information Processing Systems*, pp. 2053–2061, 2015.

Yarin Gal. A theoretically grounded application of dropout in recurrent neural networks. *arXiv preprint arXiv:1512.05287*, 2015.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Hakan Inan and Khashayar Khosravi. Improved learning through augmenting the loss. *Stanford CS 224D: Deep Learning for Natural Language Processing, Spring 2016*, 2016.

Kazuki Irie, Zoltán Tüske, Tamer Alkhouli, Ralf Schlüter, and Hermann Ney. Lstm, gru, highway and a bit of attention: an empirical overview for language modeling in speech recognition. *Interspeech, San Francisco, CA, USA*, 2016.

Camille Jordan. Essai sur la géométrie à n dimensions. *Bulletin de la Société mathématique de France*, 3:103–174, 1875.

Yoon Kim, Yacine Jernite, David Sontag, and Alexander M Rush. Character-aware neural language models. *arXiv preprint arXiv:1508.06615*, 2015.

Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Tomas Mikolov and Geoffrey Zweig. Context dependent recurrent neural network language model.

Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernockỳ, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Interspeech*, volume 2, pp. 3, 2010.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pp. 3111–3119, 2013.

Andriy Mnih and Geoffrey Hinton. Three new graphical models for statistical language modelling. In *Proceedings of the 24th international conference on Machine learning*, pp. 641–648. ACM, 2007.

Ramesh Nallapati, Bowen Zhou, Çaglar Gulçehre, and Bing Xiang. Abstractive text summarization using sequence-to-sequence rnns and beyond. 2016.

参考文献

Yoshua Bengio, R´ejean Ducharme, 和 Pascal Vincent。一种神经概率语言模型。2001年。网址 http://www.iro.umontreal.ca/~lisa/pointeurs/nips00_lm.ps。

Åke Björck and Gene H Golub. Numerical methods for computing angles between linear subspaces. *Mathematics of computation*, 27(123):579–594, 1973.

Wei-Chen Cheng, Stanley Kok, Hoai Vu Pham, Hai Leong Chieu, and Kian Ming Adam Chai. Language modeling with sum-product networks. 2014.

Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.

Orhan Firat, Kyunghyun Cho, and Yoshua Bengio. Multi-way, multilingual neural machine translation with a shared attention mechanism. *arXiv preprint arXiv:1601.01073*, 2016.

Charlie Frogner, Chiyuan Zhang, Hossein Mobahi, Mauricio Araya, and Tomaso A Poggio. Learning with a wasserstein loss. In *Advances in Neural Information Processing Systems*, pp. 2053–2061, 2015.

Yarin Gal. A theoretically grounded application of dropout in recurrent neural networks. *arXiv preprint arXiv:1512.05287*, 2015.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Hakan Inan and Khashayar Khosravi. Improved learning through augmenting the loss. *Stanford CS 224D: Deep Learning for Natural Language Processing, Spring 2016*, 2016.

Kazuki Irie, Zoltán Tüske, Tamer Alkhouli, Ralf Schlüter, and Hermann Ney. Lstm, gru, highway and a bit of attention: an empirical overview for language modeling in speech recognition. *Interspeech, San Francisco, CA, USA*, 2016.

Camille Jordan. Essai sur la géométrie à n dimensions. *Bulletin de la Société mathématique de France*, 3:103–174, 1875.

Yoon Kim, Yacine Jernite, David Sontag, and Alexander M Rush. Character-aware neural language models. *arXiv preprint arXiv:1508.06615*, 2015.

Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Tomas Mikolov and Geoffrey Zweig. Context dependent recurrent neural network language model.

Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernockỳ, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Interspeech*, volume 2, pp. 3, 2010.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pp. 3111–3119, 2013.

Andriy Mnih and Geoffrey Hinton. Three new graphical models for statistical language modelling. In *Proceedings of the 24th international conference on Machine learning*, pp. 641–648. ACM, 2007.

Ramesh Nallapati, Bowen Zhou, Çaglar Gulçehre, and Bing Xiang. Abstractive text summarization using sequence-to-sequence rnns and beyond. 2016.

Razvan Pascanu, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. How to construct deep recurrent neural networks. *CoRR*, abs/1312.6026, 2013a.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. *ICML (3)*, 28:1310–1318, 2013b.

Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *EMNLP*, volume 14, pp. 1532–43, 2014.

Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

Alexander M Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. *arXiv preprint arXiv:1509.00685*, 2015.

Richard Socher, Alex Perelygin, Jean Y Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. Citeseer, 2013.

Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.

Julian Georg Zilly, Rupesh Kumar Srivastava, Jan Koutník, and Jürgen Schmidhuber. Recurrent highway networks. *arXiv preprint arXiv:1607.03474*, 2016.

Razvan Pascanu, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. How to construct deep recurrent neural networks. *CoRR*, abs/1312.6026, 2013a.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. *ICML (3)*, 28:1310–1318, 2013b.

Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *EMNLP*, volume 14, pp. 1532–43, 2014.

Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

Alexander M Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. *arXiv preprint arXiv:1509.00685*, 2015.

Richard Socher, Alex Perelygin, Jean Y Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. Citeseer, 2013.

Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.

Julian Georg Zilly, Rupesh Kumar Srivastava, Jan Koutník, and Jürgen Schmidhuber. Recurrent highway networks. *arXiv preprint arXiv:1607.03474*, 2016.

APPENDIX

A MODEL AND TRAINING DETAILS

We begin training with a learning rate of 1 and start decaying it with a constant rate after a certain epoch. This is 5, 10, and 1 for the small, medium, and large networks respectively. The decay rate is 0.9 for the small and medium networks, and 0.97 for the large network.

For both PTB and Wikitext-2 datasets, we unroll the network for 35 steps for backpropagation.

We use gradient clipping (Pascanu et al., 2013b); i.e. we rescale the gradients using the global norm if it exceeds a certain value. For both datasets, this is 5 for the small and the medium network, and 6 for the large network.

We use the dropout method introduced in Gal (2015); particularly, we use the same dropout mask for each example through the unrolled network. Differently from what was proposed in Gal (2015), we tie the dropout weights for hidden states further, and we use the same mask when they are propagated as states in the current layer and when they are used as inputs for the next layer. We don't use dropout in the input embedding layer, and we use the same dropout probability for inputs and hidden states. For PTB, dropout probabilities are 0.7, 0.5 and 0.35 for small, medium and large networks respectively. For Wikitext-2, probabilities are 0.8 for the small and 0.6 for the medium networks.

When training the networks with the augmented loss (AL), we use a temperature $\tau = 20$. We have empirically observed that setting α, the weight of the augmented loss, according to $\alpha = \gamma\tau$ for all the networks works satisfactorily. We set γ to values between 0.5 and 0.8 for the PTB dataset, and between 1.0 and 1.5 for the Wikitext-2 dataset. We would like to note that we have not observed sudden deteriorations in the performance with respect to moderate variations in either τ or α.

B METRIC FOR CALCULATING SUBSPACE DISTANCES

In this section, we detail the metric used for computing the subspace distance between two matrices. The computed metric is closely related with the principle angles between subspaces, first defined in Jordan (1875).

Our aim is to compute a metric distance between two given matrices, X and Y. We do this in three steps:

(1) Obtain two matrices with orthonormal columns, U and V, such that span(U)=span(X) and span(V)=span(Y). U and V could be obtained with a QR decomposition.

(2) Calculate the projection of either one of U and V onto the other; e.g. do $S = UU^TV$, where S is the projection of V onto U. Then calculate the residual matrix as $R = V - S$.

(3) Let $\|.\|_{Fr}$ denote the frobenious norm, and let C be the number of columns of R. Then the distance metric is found as d where $d^2 = \frac{1}{C}\|R\|_{Fr}^2 = \frac{1}{C}\text{Trace}(R^TR)$.

We note that d as calculated above is a valid metric up to the equivalence set of matrices which span the same column space, although we are not going to show it. Instead, we will mention some metric properties of d, and relate it to the principal angles between the subspaces. We first work out an expression for d:

附录

模型与训练详情

我们以1作为学习率开始训练，且在达到特定训练轮次后以恒定速率逐步降低该值。对于小型、中型和大型网络，这一数值分别对应5、10和1。小型和中型网络的衰减率为0.9，而大型网络的衰减率为0.97。

对于PTB数据集和WIKIText-2数据集，我们均将网络展开35步以进行反向传播。

我们采用梯度裁剪（Pascanu等人，2013b）；也就是说，当梯度的全局范数超过特定数值时，我们会对其重新进行缩放。对于这两个数据集，该数值分别为5，适用于小型和中等规模的网络，而6则适用于大型网络。

We use the dropout method introduced in Gal (2015); particularly, we use the same dropout mask for each example through the unrolled network. Differently from what was proposed in Gal (2015), we tie the dropout weights for hidden states further, and we use the same mask when they are propagated as states in the current layer and when they are used as inputs for the next layer. We don't use dropout in the input embedding layer, and we use the same dropout probability for inputs and hidden states. For PTB, dropout probabilities are 0.7, 0.5 and 0.35 for small, medium and large networks respectively. For Wikitext-2, probabilities are 0.8 for the small and 0.6 for the medium networks.

在利用增强损失（AL）对网络进行训练时，我们会使用一个温度参数 $\tau = 20$。通过实验我们发现，对于所有网络而言，按照 $\alpha = \gamma\tau$ 设定增强损失的权重 α，就能获得令人满意的效果。针对PTB数据集，我们将 γ 的数值设定在0.5到0.8之间；而对于Wikitext-2数据集，则将其设定在1.0到1.5之间。需要特别说明的是，即便 τ 或 α发生适度变化，我们也未观察到模型性能出现突然下降的情况。

用于计算子空间距离的B度量

在本节中，我们将详细阐述用于计算两个矩阵之间子空间距离的度量方法。该度量值与子空间之间的主角度密切相关，而这些主角度最早是在Jordan (1875)年中定义的。

Our 其目的是计算两个给定矩阵 X 和 Y之间的度量距离。我们通过以下方式来实现这一点：
前几步

(1) 获取两个具有正交归一化列的矩阵 U和 V，使得span(U)=span(X)且span(V)=span(Y)。 U和 V可通过QR分解获得。(2) 计算 U与 V中任意一个向另一个的投影；例如计算 $S = UU^TV$，其中 S为 V在 U上的投影。随后将残差矩阵定义为 $R = V - S$。(3) 令 $\|.\|_{Fr}$表示弗罗贝尼乌斯范数， C表示 R的列数。此时距离度量可表示为 d，其中$d^2 = \frac{1}{C}\|R\|_{Fr}^2 = \frac{1}{C}$为Trace($R^TR$)。

需要说明的是，如上所述计算得到的 d 在那些张成相同列空间的矩阵等价类范围内确实属于有效的度量，尽管我们此处不会对此进行证明。相反，我们将介绍 d的一些度量特性，并将其与子空间之间的主角度联系起来。首先，我们来推导 d的表达式：

$$
\begin{aligned}
Cd^2 &= \text{Trace}(R^T R) = \text{Trace}\left((V - UU^T V)^T (V - UU^T V)\right) \\
&= \text{Trace}\left(V^T (I - UU^T)(I - UU^T) V\right) \\
&= \text{Trace}\left(V^T (I - UU^T) V\right) \\
&= \text{Trace}\left((I - UU^T) V V^T\right) \\
&= \text{Trace}(V^T V) - \text{Trace}\left(UU^T V V^T\right) \\
&= C - \text{Trace}\left(UU^T V V^T\right) \\
&= C - \text{Trace}\left((U^T V)^T (U^T V)\right) \\
&= C - \|U^T V\|_{Fr}^2 \\
&= \sum_{i=1}^{C} 1 - \rho_i^2, \qquad \text{(B.1)}
\end{aligned}
$$

where ρ_i is the ith singular value of $U^T V$, commonly referred to as the ith principle angle between the subspaces of X and Y, θ_i. In above, we used the cyclic permutation property of the trace in the third and the fourth lines.

Since d^2 is $\frac{1}{C}\text{Trace}(R^T R)$, it is always nonnegative, and it is only zero when the residual is zero, which is the case when span(X) = span(Y). Further, it is symmetric between U and V due to the form of (B.1) (singular values of $V^T U$ and $V^T U$ are the same). Also, $d^2 = \frac{1}{C}\sum_{i=1}^{C} \sin^2(\theta_i)$, namely the average of the sines of the principle angles, which is a quantity between 0 and 1.

$$
\begin{aligned}
Cd^2 &= \text{Trace}(R^T R) = \text{Trace}\left((V - UU^T V)^T (V - UU^T V)\right) \\
&= \text{Trace}\left(V^T (I - UU^T)(I - UU^T) V\right) \\
&= \text{Trace}\left(V^T (I - UU^T) V\right) \\
&= \text{Trace}\left((I - UU^T) V V^T\right) \\
&= \text{Trace}(V^T V) - \text{Trace}\left(UU^T V V^T\right) \\
&= C - \text{Trace}\left(UU^T V V^T\right) \\
&= C - \text{Trace}\left((U^T V)^T (U^T V)\right) \\
&= C - \|U^T V\|_{Fr}^2 \\
&= \sum_{i=1}^{C} 1 - \rho_i^2, \qquad \text{(B.1)}
\end{aligned}
$$

其中 ρ_i 是 $U^T V$的 i^{th} 奇异值，通常被称作 X 与 Y、 θ_i所对应子空间之间的 i^{th} 主角度。在上述推导过程中，第三行和第四行运用了迹的循环置换性质。

Since d^2 is $\frac{1}{C}\text{Trace}(R^T R)$, it is always nonnegative, and it is only zero when the residual is zero, which is the case when span(X) = span(Y). Further, it is symmetric between U and V due to the form of (B.1) (singular values of $V^T U$ and $V^T U$ are the same). Also, $d^2 = \frac{1}{C}\sum_{i=1}^{C} \sin^2(\theta_i)$, namely the average of the sines of the principle angles, which is a quantity between 0 and 1.